

ROS parameter

Prof.: Dr. Oybek Eraliev

TA: Jasurbek Mamurov

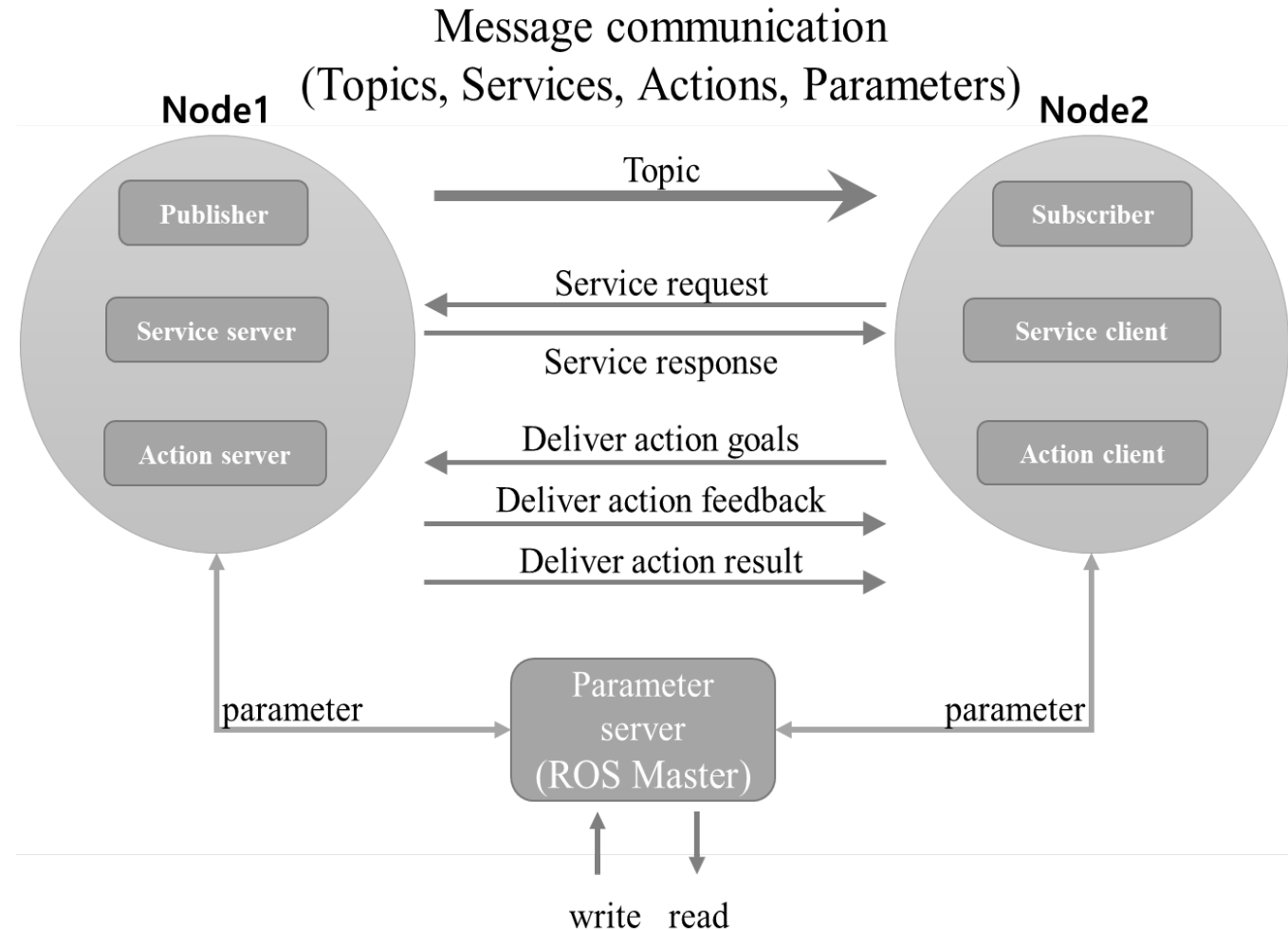


Content:

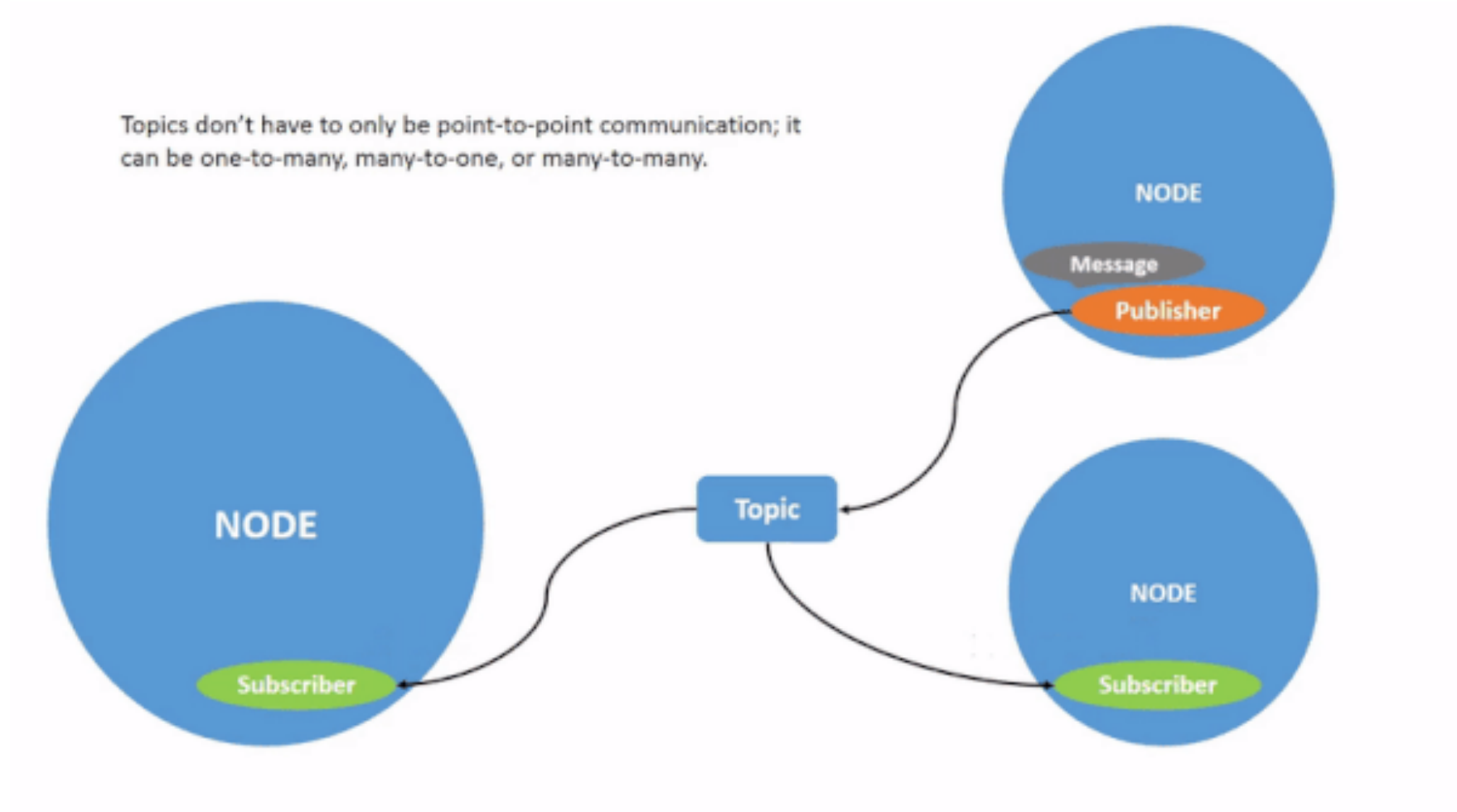
- Review
- How to use parameters



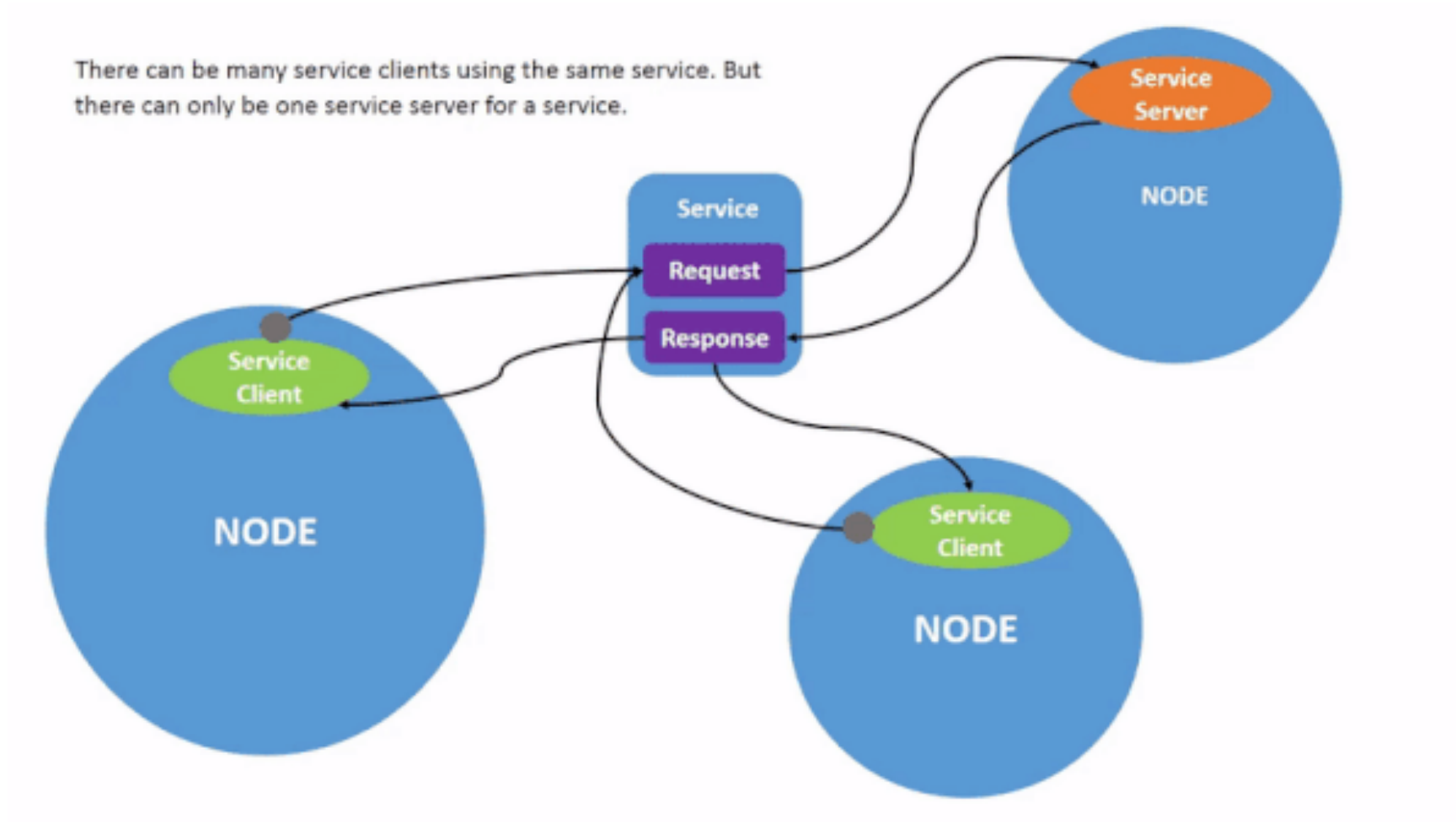
ROS Message communication



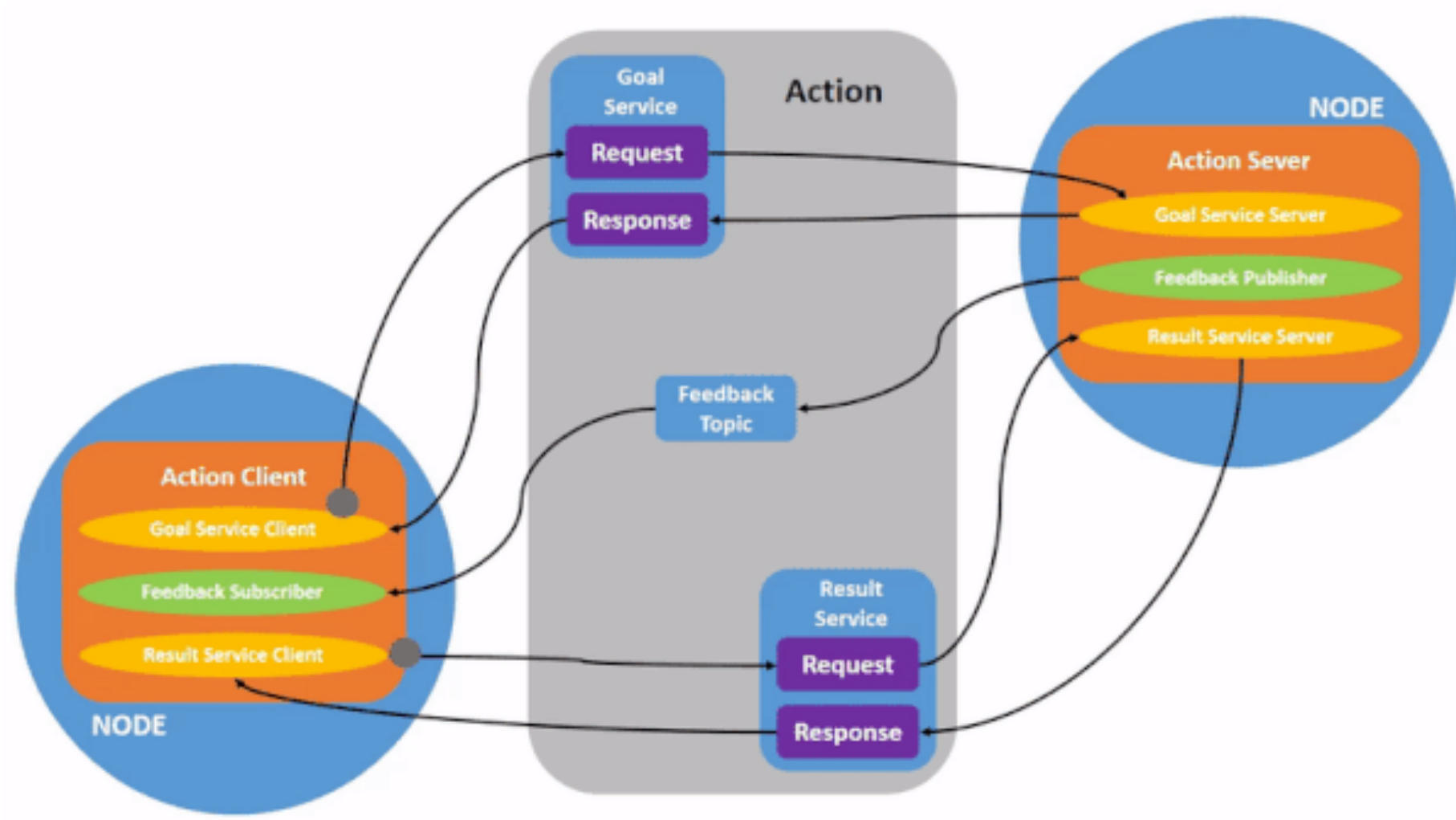
Review Topic



Review Service



Action



Content:

- ~~Review~~
- How to use parameters



Parameter

The Problem Without Parameters

- Imagine you have a robot. Without parameters you must:
- Change the code, recompile, and restart just to change the robot speed
- Maintain separate executables for different configurations
- Cannot update settings while the robot is running

The Solution: ROS Parameter Server

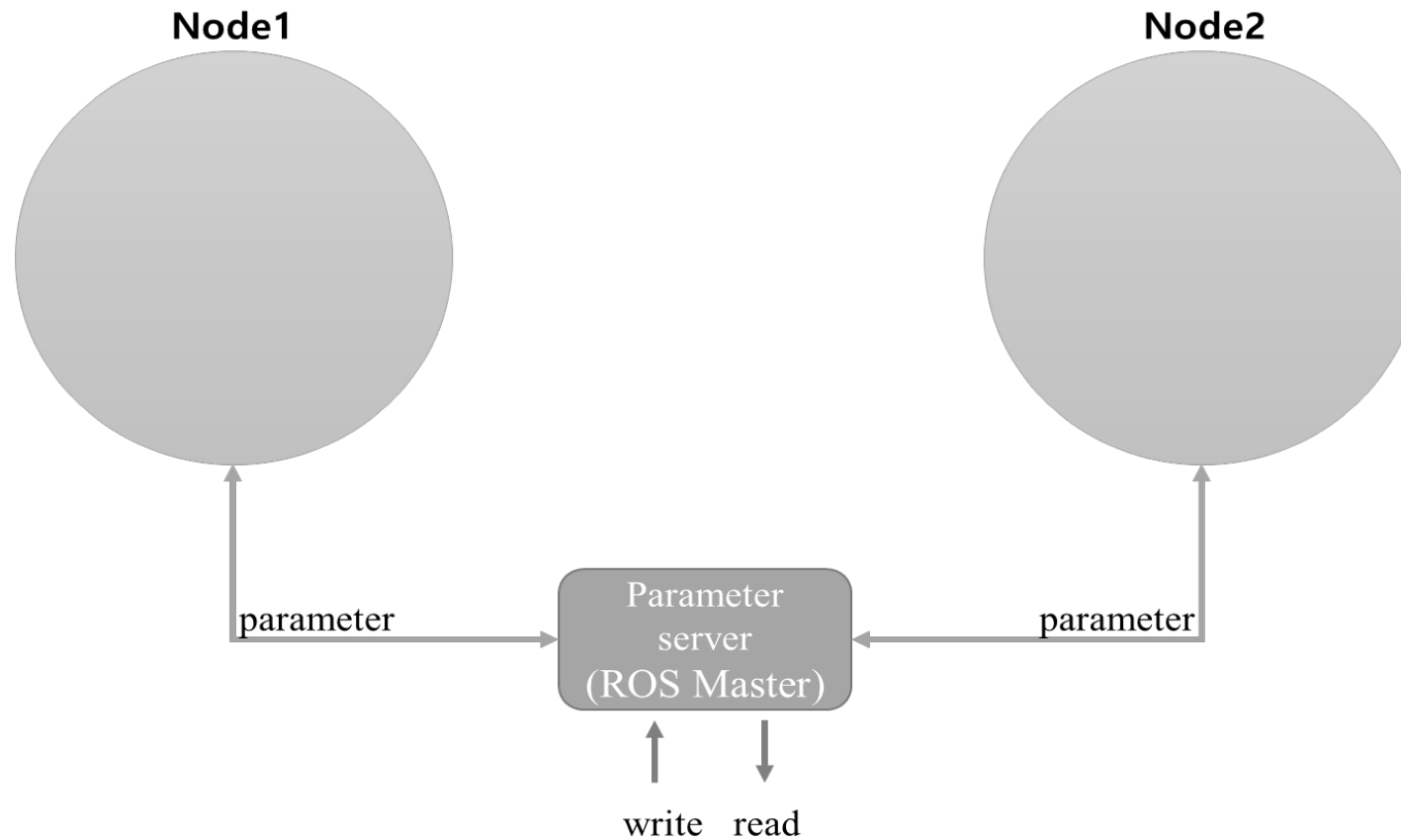
- A shared dictionary (key-value store) hosted by ROS Master
- Any node can read or write parameters at any time
- Change robot speed via terminal — no recompile needed!
- Save and load configurations to/from YAML files



Real-World Example

Change the camera frame rate,
PID gains, or map resolution
— without stopping the robot!

Parameter



Parameter

ROS parameters support the following data types. Always match your type when getting/setting.

Type	Description	Python Example
bool	True / False	<code>rospy.set_param('/flag', True)</code>
int	Whole numbers	<code>rospy.set_param('/speed', 10)</code>
double	Decimal numbers	<code>rospy.set_param('/gain', 0.75)</code>
string	Text values	<code>rospy.set_param('/mode', 'auto')</code>
list	Array of values	<code>rospy.set_param('/joints', [1,2,3])</code>
dictionary	Key-value nested structure	<code>rospy.set_param('/robot', { 'speed':1, 'name':'r2' })</code>

Parameter

- 1. Create nodes using parameters
- Let's create a **service_server.py** and client node to use parameters to perform arithmetic operations, rather than merely adding a and b entered as service requests.

```
1  #!/usr/bin/env python3
2
3  import rospy
4  from beginner_tutorials.srv import SrvTutorial, SrvTutorialResponse
5
6  # Define operation codes
7  PLUS = 1
8  MINUS = 2
9  MULTIPLICATION = 3
10 DIVISION = 4
11
12 # Global variable to store operation mode
13 g_operator = PLUS
14
15 def calculation(req):
16     """Callback function to perform arithmetic operations based on parameters"""
17     global g_operator
18
19     # Perform the requested arithmetic operation
20     if g_operator == PLUS:
21         res = req.a + req.b
22     elif g_operator == MINUS:
23         res = req.a - req.b
24     elif g_operator == MULTIPLICATION:
25         res = req.a * req.b
26     elif g_operator == DIVISION:
27         res = 0 if req.b == 0 else req.a // req.b
28     else:
29         res = req.a + req.b # Default case
30
31     # Log request and response
32     rospy.loginfo("Request received: x=%d, y=%d", req.a, req.b)
33     rospy.loginfo("Sending back response: [%d]", res)
34
35     return SrvTutorialResponse(res)
```

Parameter

```
37  if __name__ == "__main__":
38      # Initialize ROS node
39      rospy.init_node("ros_tutorial_service")
40
41      # Set initial parameter for calculation method
42      rospy.set_param("/calculation_method", PLUS)
43
44      # Create a service server
45      service = rospy.Service("ros_tutorial_srv", SrvTutorial, calculation)
46      rospy.loginfo("Ready to serve requests!")
47
48      # Maintain loop to update the operation mode from parameters
49      rate = rospy.Rate(10) # 10 Hz
50      while not rospy.is_shutdown():
51          g_operator = rospy.get_param("/calculation_method", PLUS)
52          rate.sleep()
53
```

Parameter

Create the **SrvTutorial.srv** file and insert the code below:

```
src > beginner_tutorials > srv > ≡ SrvTutorial.srv
1  int64 a
2  int64 b
3  ---
4  int64 rusult
5
```

Parameter: Live Usage Example

Set parameters from the terminal while the service server is running and watch behaviour change:

```
# Default: addition (method=1)
$ rosservice call /ros_tutorial_srv 10 5
  result: 15

$ roseth param set /calculation_method 2 # subtract
$ rosservice call /ros_tutorial_srv 10 5
  result: 5

$ roseth param set /calculation_method 3 # multiply
$ rosservice call /ros_tutorial_srv 10 5
  result: 50

$ roseth param set /calculation_method 4 # divide
$ rosservice call /ros_tutorial_srv 10 5
  result: 2
```

Key Insight

The node did NOT restart.

The server is polling the parameter server at 10 Hz and updates its operation mode automatically.

This is the power of ROS Parameters!

rosparam Commands (Complete Reference)

All rosparam commands — use these from the terminal while your nodes are running:

```
# List all parameters currently on the parameter server
$ rosparam list

# Get the value of a specific parameter
$ rosparam get /calculation_method
1

# Set a parameter value (change operation to subtract=2)
$ rosparam set /calculation_method 2

# Save ALL current parameters to a YAML file
$ rosparam dump config.yaml

# Load parameters from a YAML file into the server
$ rosparam load config.yaml

# Delete a parameter from the server
$ rosparam delete /calculation_method
```

```
# Python API equivalents:
rospy.set_param('/key', value)

# With default fallback:
val = rospy.get_param(
    '/key', default_val)

# Check if param exists:
if rospy.has_param('/key'):
    ...

# Delete from code:
rospy.delete_param('/key')

# ⚠ Params are LOST when
#   roscore shuts down!
#   Use YAML files to persist.
```

rosparam Commands (Complete Reference)

All rosparam commands — use these from the terminal while your nodes are running:

```
# Run turtles node and teleop_key node
roslaunch turtlesim turtlesim_node
roslaunch turtlesim turtle_teleop_key

# Set the RGB values (0-255)
rosparam set /turtlesim/background_r 255
rosparam set /turtlesim/background_g 0
rosparam set /turtlesim/background_b 0

# Then call /clear to apply the change
rosservice call /clear
```

```
# Spawn a second turtle at position x=5, y=5,
angle=0, named "turtle2"

rosservice call /spawn 5 5 0 "turtle2"

# Move turtle2 specifically

rostopic pub /turtle2/cmd_vel geometry_msgs/
Twist \
    '{linear: {x: 2.0}, angular: {z: 1.0}}'

# Kill a turtle by name
rosservice call /kill "turtle2"

# Teleport turtle1 instantly
rosservice call /turtle1/teleport_absolute 5 5 0

# Teleport relative to current position
rosservice call /turtle1/teleport_relative 1.0
0.5
```

Parameters: YAML File Workflow

Save and reload configurations with YAML files — essential for real robot deployments:

config/robot_params.yaml

```
# robot_params.yaml
calculation_method: 1
robot:
  max_speed: 1.5
  name: 'turtlebot3'
  sensors:
    - lidar
    - camera
```

Terminal commands

```
# Save all params to YAML
$ rosparam dump config/robot_params.yaml

# Load params from YAML file
$ rosparam load config/robot_params.yaml

# Load inside launch file:
<rosparam file="config/robot_params.yaml"
  command="load"/>
```

```
#  Important: Param server is tied to roscore.
# When roscore stops, ALL params are LOST!
# Always save to YAML before shutdown.
```

Live Parameter Tuning: rqt_reconfigure

rqt_reconfigure is a GUI tool that lets you adjust parameters with sliders and dropdowns — in real time — without touching the terminal.

```
# Step 1: Install (if not present)
$ sudo apt-get install ros-noetic-rqt-reconfigure

# Step 2: Launch the GUI
$ rosrun rqt_reconfigure rqt_reconfigure

# Step 3: Your node must use dynamic_reconfigure
from dynamic_reconfigure.server import Server
from your_pkg.cfg import TutorialConfig
```

Great For:



PID gain tuning



Camera exposure / FPS



Map resolution



Robot max speed / turn rate



Any numeric or boolean param

Note: rqt_reconfigure requires nodes to implement the dynamic_reconfigure package. For simple use cases, rosparam set via terminal is sufficient.

Parameter

- We have already discussed how to modify the existing service server and use parameters.
- The related source has been renamed `ros_tutorials_parameter` package in order to distinguish it from the service source code that was created previously, and can be found at the github address below.
- https://github.com/ROBOTIS-GIT/ros_tutorials/tree/master/ros_tutorials_parameter
- If you want to apply it right away, you can clone the source code with the following command in the `catkin_ws / src` folder and run the build. Then run the `service_server` and `service_client` nodes.

```
$ cd ~/catkin_ws/src
```

```
$ git clone https://github.com/ROBOTIS-GIT/ros_tutorials.git
```

```
$ cd ~/catkin_ws
```

```
$ catkin_make
```

```
$ rosrun ros_tutorials_parameter service_server_with_parameter
```

```
$ rosrun ros_tutorials_parameter service_client_with_parameter 2 3
```



Thank you!

